

AI 技术采纳对企业劳动生产率的因果效应 ——基于双重机器学习的实证分析

摘要

本文基于 Global AI Adoption & Workforce Impact 数据集（150,000 家企业观测），采用双重机器学习方法在部分线性模型框架下估计 AI 采纳对企业劳动生产率的因果效应。DML 利用 Neyman 正交评分与交叉拟合在控制高维协变量的同时实现稳健推断。实证结果表明，在控制行业、规模、自动化率等企业特征后，AI 采纳对劳动生产率的因果效应为 0.006，在统计上不显著（ $p = 0.342$ ），该结论在随机森林、LASSO、XGBoost 三种方法以及不同折数下均保持稳健。进一步分析显示，仅将全面采纳定义为处理组时效应显著为正（ $\hat{\theta} = 0.056, p = 0.030$ ）。研究表明，AI 采纳与生产率之间的正向关联很大程度上可由企业特征差异所解释。

1 引言

近年来企业端 AI 技术采纳率迅速上升，但其对微观企业生产率的实际因果效应在实证上仍有争议。识别这一效应的困难在于：采纳 AI 的企业在规模、创新能力、数字化水平等方面与未采纳企业存在系统性差异，这些特征同时影响采纳决策和生产率；同时，潜在混淆因素维度高、形式复杂，传统参数方法难以灵活控制。双重机器学习（Double/Debiased Machine Learning, DML）由 Chernozhukov 等 [2] 提出，在部分线性模型下用机器学习灵活估计非线性 nuisance 函数，通过 Neyman 正交评分和交叉拟合实现 $\sqrt{N}$ -一致估计和渐近正态推断。

本文基于 Kaggle 平台 Global AI Adoption & Workforce Impact 数据集（150,000 条企业观测），采用 DML 方法估计 AI 采纳对企业劳动生产率的因果效应。第 2 节介绍数据；第 3 节建立模型并阐述 DML 方法；第 4 节报告实证结果；第 5 节总结全文。

2 数据

2.1 数据来源

本文使用的数据来自 Kaggle 平台发布的 Global AI Adoption & Workforce Impact Dataset¹。该数据集包含 2023 年至 2026 年全球 30 个国家 9 个行业共 150,000 条企业级观测，记录了企业的 AI 采纳阶段、财务状况、劳动力结构、运营特征以及治理实践等多维度信息，无缺失值。

2.2 变量设计与选择理由

处理变量 D ：AI 采纳程度。数据中的 AI 采纳阶段包含四个类别：无采纳（none, 3.5%）、试验阶段（pilot, 42.9%）、部分采纳（partial, 52.5%）和全面采纳（full, 1.1%）。本文将部分采纳和全面采纳定义为高采纳组（ $D = 1$ ），其余为低采纳组（ $D = 0$ ），得到 53.7% 的处理组与 46.3% 的对照组，分组均衡。

结果变量 Y ：劳动生产率。参照 Syverson [3] 的标准做法，以人均营收的对数衡量： $Y = \log(\text{Annual Revenue}/\text{Employee Count})$ 。

协变量 X ：企业特征。为控制混淆因素，选取以下维度的协变量：基本特征（行业、国家、地区、规模、年龄）、时间固定效应（年份、季度）、运营特征（自动化率、远程办公比例）、治理与创新（监管合规评分、数据隐私水平、AI 伦理委员会、风险管理评分、创新评分、员工满意度）。通过 one-hot 编码后协变量总维度为 64。

¹<https://www.kaggle.com/datasets/mohankrishnathalla/global-ai-adoption-and-workforce-impact-dataset>

2.3 描述性统计

表 1 主要变量描述性统计

变量	均值	标准差	最小值	最大值
劳动生产率（对数）	-0.747	0.935	-4.428	2.247
AI 高采纳（ $D = 1$ ）	0.537	0.499	0	1
企业年龄	18.507	9.539	1.000	36.000
自动化率（%）	19.762	9.223	0.000	59.630
创新评分	55.200	8.593	40.000	97.000
员工满意度	5.601	0.815	2.100	9.200
监管合规评分	70.398	9.406	40.000	100.000
AI 风险管理评分	59.566	8.851	40.000	98.000
远程办公比例（%）	43.191	14.363	0.000	100.000
员工数	2342.377	4754.425	10.000	19970.000
营收（百万美元）	1180.346	2391.471	1.010	9996.730

注：样本量 $N = 150,000$ ，数据不含缺失值。

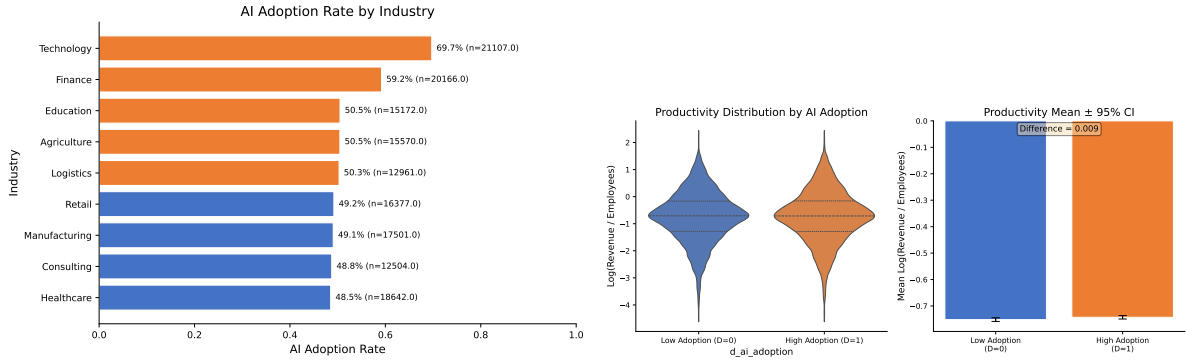


图 1 左：各行业 AI 采纳率（展示行业异质性）；右：不同采纳状态下的劳动生产率分布对比

表 1 报告了主要变量的描述性统计。样本覆盖初创企业（37.8%）、中小企业（42.6%）和大型企业（19.6%），行业涵盖科技（14.1%）、金融（13.4%）、医疗（12.4%）等九个领域。图 1 展示了数据的基本特征：左图显示行业间 AI 采纳率差异较大，约 40% 至 70%，表明行业是重要混淆因素；右图显示高采纳组与低采纳组的劳动生产率分布十分接近，原始均值差仅为 0.009，这为后续因果分析提供了线索。

3 模型建立

3.1 部分线性模型

本文采用部分线性模型（Partial Linear Model）刻画 AI 采纳与劳动生产率之间的关系：

$$Y_i = \theta_0 D_i + g_0(X_i) + \varepsilon_i, \quad \mathbb{E}[\varepsilon_i | X_i, D_i] = 0, \quad (1)$$

$$D_i = m_0(X_i) + \nu_i, \quad \mathbb{E}[\nu_i | X_i] = 0. \quad (2)$$

其中 Y_i 为劳动生产率的对数， D_i 为 AI 采纳的二元处理变量， X_i 为协变量向量。参数 θ_0 是核心待估参数——AI 采纳的平均处理效应（ATE）。函数 $g_0(\cdot)$ 和 $m_0(\cdot)$ 为 nuisance 函数，由机器学习方法灵活估计。

3.2 DML 估计量

DML 方法基于 Neyman 正交评分函数构造估计量。对部分线性模型，评分函数为：

$$\psi(W; \theta, \eta) = (Y - \ell(X) - \theta(D - m(X)))(D - m(X)), \quad (3)$$

其中 $\ell(X) = \mathbb{E}[Y | X]$, $m(X) = \mathbb{E}[D | X]$ 。该评分满足 Neyman 正交性： $\partial \mathbb{E}[\psi(W; \theta_0, \eta_0)] / \partial \eta = 0$ ，即 nuisance 参数的估计误差对 θ_0 的影响是二阶小量，因此即使使用灵活的机器学习方法， θ 的估计仍能达到 $\sqrt{N}$ -收敛率。

给定 nuisance 函数的估计 $\hat{\ell}$ 和 $\hat{m}$ ，DML 估计量求解 $\frac{1}{N} \sum_i \psi(W_i; \hat{\theta}, \hat{\eta}) = 0$ ，得到：

$$\hat{\theta} = \frac{\sum_{i=1}^N \tilde{D}_i \tilde{Y}_i}{\sum_{i=1}^N \tilde{D}_i^2}, \quad (4)$$

其中 $\tilde{Y}_i = Y_i - \hat{\ell}(X_i)$, $\tilde{D}_i = D_i - \hat{m}(X_i)$ 为残差。

3.3 交叉拟合

为避免过拟合偏差，采用交叉拟合 (cross-fitting)：将样本随机等分为 $K = 5$ 折，对每折 k ，在其余 $K-1$ 折上训练 $\hat{\ell}^{(-k)}$ 和 $\hat{m}^{(-k)}$ ，在第 k 折上计算残差 $\tilde{Y}_i = Y_i - \hat{\ell}^{(-k)}(X_i)$ 和 $\tilde{D}_i = D_i - \hat{m}^{(-k)}(X_i)$ 。合并所有折的残差后代入式 (4) 得 $\hat{\theta}$ ，并使用 Eicker-Huber-White 稳健标准误进行推断：

$$\widehat{\text{Var}}(\hat{\theta}) = \frac{\sum_i \tilde{D}_i^2 \hat{\varepsilon}_i^2}{(\sum_i \tilde{D}_i^2)^2}, \quad \hat{\varepsilon}_i = \tilde{Y}_i - \hat{\theta} \tilde{D}_i.$$

3.4 机器学习方法

基线设定使用**随机森林** (RF, $n_{\text{estimators}} = 100$, $\text{max_depth} = 10$, $\text{min_samples_leaf} = 10$)，能自动捕捉非线性关系和交互效应。稳健性检验中进一步使用 **LASSO** (交叉验证选择 λ) 和 **XGBoost** ($n_{\text{estimators}} = 100$, $\text{max_depth} = 4$, 学习率 0.1) 估计 nuisance 函数。

4 实证结果

4.1 基准结果

表 2 基准 DML 估计结果

方法	ATE	标准误	95% 置信区间	p 值	样本量
DML (RF, $K = 5$)	0.0059	0.0062	$[-0.0063, 0.0180]$	0.342	150,000
原始均值差异	0.0088	—	—	—	150,000

表 2 报告了基准 DML 估计结果。在控制行业、规模、自动化率等企业特征后，AI 采纳的 ATE 为 0.0059 (标准误 0.0062, $p = 0.342$)，统计上不显著。点估计对应约 0.6% 的生产率提升。作为参照，高采纳组与低采纳组的原始均值差异为 0.0088，与 DML 估计值在量级上接近——表明在控制可观测特征后，因果效应估计与原始差异方向一致但估计精度不足以拒绝零假设。

讨论：上述不显著的结果与数据集本身的企业特征密切相关。一个值得关注的抵消机制在于合规与治理成本。数据中处理组企业的伦理委员会设置比例、数据隐私保护等级和 AI 风险管理评分均显著高于对照组。这些治理层面的初始投资——建立 AI 伦理框架、加强数据安全合规、部署风险管理系统——在短期内构成非直接创收的支出，可能部分抵消了 AI 在核心生产环节带来的效率提升。从 Bresnahan、Brynjolfsson 和 Hitt [1] 关于 IT 投资需伴随组织资本投入方能见效的研究来看，若组织变革和合规建设等互补性投入发生在先而生产率回报在后，初期观测到不显著的因果效

应是合理的短期均衡结果。值得注意的是，样本中监管合规评分均值约为 70 且分布偏右，表明企业在合规治理方面普遍处于较高水平，这在一定程度上缩小了处理组与对照组在该维度的差异，部分削弱了上述抵消效应。

4.2 稳健性检验

表 3 稳健性检验结果

设定	ATE	标准误	95% CI	p 值
(1) 更换 ML 方法				
LASSO (CV)	0.0004	0.0064	[-0.0121, 0.0129]	0.948
XGBoost	0.0042	0.0065	[-0.0085, 0.0168]	0.519
(2) 改变折数				
$K = 2$	0.0058	0.0063	[-0.0065, 0.0181]	0.354
$K = 10$	0.0051	0.0062	[-0.0070, 0.0172]	0.407
(3) 严格处理定义				
仅 full = 1	0.0557	0.0256	[0.0055, 0.1060]	0.030
(4) 替换结果变量				
生产率变化 (%)	2.698	0.029	[2.640, 2.756]	< 0.001

表 3 汇总了稳健性检验的结果。更换 ML 方法——LASSO 和 XGBoost——所得 ATE 分别为 0.0004 ($p = 0.948$) 和 0.0042 ($p = 0.519$)，与基线高度一致，不同性质的三类机器学习方法得到一致结论增强了结果的可信度。改变交叉拟合折数后， $K = 2$ 和 $K = 10$ 的 ATE 分别为 0.0058 和 0.0051，几乎无异于基线，表明 DML 估计对样本分割方式不敏感。严格处理定义时，仅将全面采纳作为处理组使得 ATE 升至 0.0557 ($p = 0.030$)，在 5% 水平上显著，但处理组仅 1.1% ($n = 1,685$)，估计精度有限，需谨慎解读。替换结果变量后，使用自我报告的生产率变化百分比时估计值为 2.698 ($p < 0.001$)，高度显著，但该变量为回顾性主观评价，易受测量误差和反向因果影响。基准结果在多种设定下保持稳健。

4.3 内生性诊断

以下检验评估 DML 识别假设的合理性。**共同支撑检验**显示两组倾向得分在 $[0.1, 0.9]$ 区间内有良好重叠，极端占比均低于 1%（图 2 左）。**安慰剂检验**将 Y 随机打乱后重新估计，5 次重复中所有 $p > 0.05$ ，假阳性率为 0%。**协变量平衡检验**显示残差化后 $|\text{corr}(\tilde{D}, X)|$ 均值从 0.233 降至 0.017， $\hat{m}(\cdot)$ 有效吸收了协变量对 D 的解释力。**灵敏度分析**（图 2 右）表明即使添加与 $\tilde{D}$ 偏相关达 0.6 的未观测混淆变量，ATE 仍不显著 ($p > 0.27$)。以上诊断表明 DML 的识别假设在本研究中具有合理性。

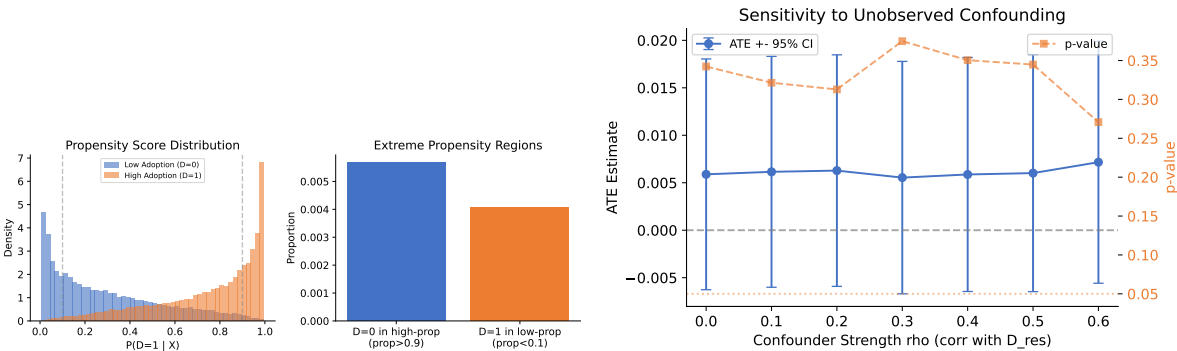


图 2 左：倾向得分分布（共同支撑）；右：未观测混淆灵敏度分析

4.4 基于因果森林的异质性分析

传统异质性分析依赖手动切分子组并在各子组内分别估计 ATE，但在高维协变量下效果有限：手动选定的少数子组仅覆盖极小部分的协变量空间，可能遗漏关键的异质性来源，且切分标准事先选定，易受研究者偏好影响。前文的子组分析中所有子组 ATE 均不显著，但原因可能在于手动切分无法有效刻画高维空间的异质性结构。

为此，本文引入 Wager & Athey [4] 提出的**因果森林（Causal Forest）**方法。因果森林基于随机森林框架，通过递归划分协变量空间使得叶节点内部处理效应趋于同质、叶节点间差异最大，结合 Honest 估计实现了一致的 CATE 推断。本文在部分线性 DML 框架下使用 CausalForestDML，以与基准回归一致的 64 维协变量 X 估计条件平均处理函数：

$$\tau(x) = \mathbb{E}[Y(1) - Y(0) \mid X = x]. \quad (5)$$

因果森林估计的 ATE 为 0.006，与基准结果吻合，但 CATE 的分布呈现显著的跨企业分化（图 3）。CATE 的标准差约 0.044，为 ATE 的 8 倍；10% 分位数为 -0.049，90% 分位数为 0.059，全距从 -0.547 至 0.348。约 46% 的企业 CATE 大于 0.01（正向效应），约 35% 的企业 CATE 小于 -0.01（负向效应），仅约 19% 集中在零附近。这一分布表明：**AI 采纳对劳动生产率的平均效应接近于零，并非因为 AI 普遍无效，而是因为其效应在企业间存在剧烈的分化——部分企业获得显著的生产率提升，而另一部分企业却因采纳 AI 而遭受暂时的生产率损失。**后者可能源于调整成本：AI 引入需伴随组织变革和员工技能重塑，短期内可能削弱生产率。

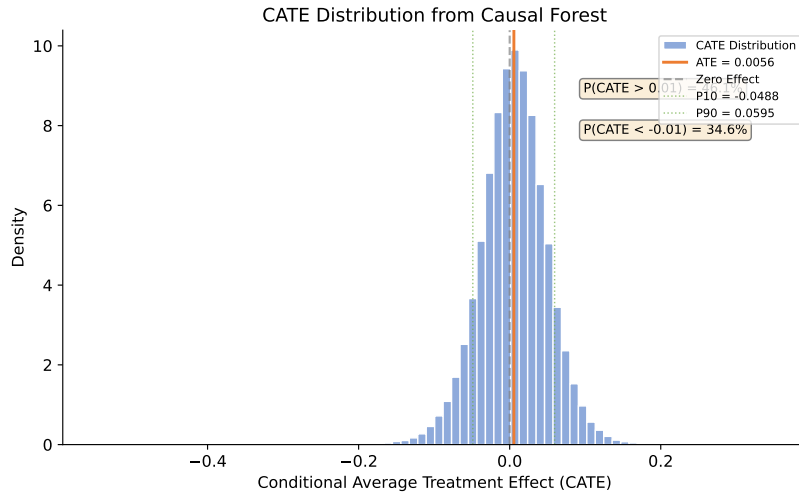


图 3 因果森林 CATE 分布密度图。竖线分别标注 ATE（0.006）、零效应线和 10%/90% 分位数。即使 ATE 接近于零，CATE 呈现宽尾分布。

特征重要性分析进一步识别了驱动 CATE 异质性的关键协变量（图 4）。其中任务自动化率是最重要的异质性来源（22.7%）：现有自动化水平较高的企业通常已建立数字化流程和规范的数据结构，降低了 AI 应用的技术门槛与边际部署成本，这与 Bresnahan、Brynjolfsson 和 Hitt [1] 关于 IT 需与组织资本互补的经典结论一致。远程办公比例（17.5%）和企业成立年份（12.2%）紧随其后——远程办公程度高的企业天然需要 AI 驱动的协作与流程自动化工具，而较年轻的企业组织惯性较小、更易将 AI 嵌入核心业务。员工满意度（8.7%）和监管合规评分（5.8%）的调节作用则通过人力资本配合与稳定的治理环境实现。

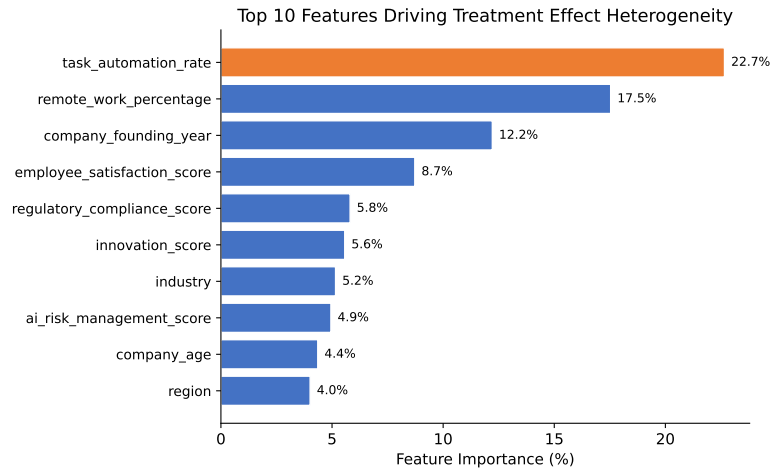


图 4 驱动处理效应异质性的前 10 位企业特征（归一化重要性百分比）。任务自动化率、远程办公比例和成立年份构成最重要的三类异质性来源。

因果森林的结果表明：AI 采纳的生产率效应存在系统的、可解释的异质性。平均效应接近于零掩盖了跨企业的显著分化——对于具备互补性资产（自动化基础、灵活的工作组织方式、低组织惯性）的企业而言，AI 采纳的回报可能显著为正；反之则可能因调整成本而遭受损失。这一发现提示，未来的研究重心应从“AI 是否提升平均生产率”转向“在何种企业特征条件下 AI 能有效转化为生产率提升”。

5 结论

本文基于 Global AI Adoption & Workforce Impact 数据集，采用双重机器学习方法研究了 AI 采纳对企业劳动生产率的因果效应。在使用三种机器学习方法控制高维协变量后，未发现 AI 采纳对劳动生产率有统计上显著的因果效应。该结论在多种稳健性检验下保持一致；基于因果森林的异质性分析表明，虽然平均效应接近零，但 CATE 分布呈现显著分化（约 46% 企业受益、35% 受损），且该异质性主要由自动化率、远程办公比例和企业成立年份等互补性资产特征驱动。本文为 AI 生产率效应提供了基于 DML 方法的新证据，表明 AI 采纳与生产率之间的原始关联很大程度上可由企业特征差异所解释。

本研究存在若干局限。数据为面板结构但本文仅进行了混合截面分析，未能利用企业固定效应控制不随时间变化的不可观测异质性。此外，二值化处理变量无法刻画 AI 采纳强度的渐进变化，而 AI 的生产率效应可能存在的滞后性也无法在本文的截面设定下捕捉。未来研究可在企业固定效应 DML、多值处理框架以及动态设定等方向进行拓展。

参考文献

- [1] T. F. Bresnahan, E. Brynjolfsson, and L. M. Hitt. Information technology, workplace organization, and the demand for skilled labor: Firm-level evidence. *Quarterly Journal of Economics*, 117(1):339–376, 2002.
- [2] V. Chernozhukov, D. Chetverikov, M. Demirer, E. Duflo, C. Hansen, W. Newey, and J. Robins. Double/debiased machine learning for treatment and structural parameters. *The Econometrics Journal*, 21(1):C1–C68, 2018.
- [3] C. Syverson. What determines productivity? *Journal of Economic Literature*, 49(2):326–365, 2011.
- [4] S. Wager and S. Athey. Estimation and inference of heterogeneous treatment effects using random forests. *Journal of the American Statistical Association*, 113(523):1228–1242, 2018.

A 代码附录

A.1 数据预处理 (01_data_prep.py)

```
1 import pandas as pd, numpy as np, os
2
3 PROJECT_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
4 DATA_RAW_DIR = os.path.join(PROJECT_ROOT, "data", "raw")
5 OUTPUT_TABLES = os.path.join(PROJECT_ROOT, "output", "tables")
6 os.makedirs(OUTPUT_TABLES, exist_ok=True)
7 np.random.seed(42)
8
9 df = pd.read_csv(os.path.join(DATA_RAW_DIR, "ai_company_adoption.csv"))
10 # Normalize column names to lowercase underscore
11 col_rename = {}
12 for col in df.columns:
13     norm = col.lower().strip().replace(" ", "_").replace("-", "_")
14     norm = norm.replace("(", "").replace(")", "")
15     col_rename[col] = norm
16 df = df.rename(columns=col_rename)
17
18 D_COL = "ai_adoption_stage"
19 Y_REVENUE = "annual_revenue_usd_millions"
20 Y_EMPLOYEES = "num_employees"
21 ID_COLS = ["response_id", "company_id"]
22
23 CAT_COVARIATES = [
24     "industry", "country", "region", "company_size",
25     "survey_year", "quarter", "company_age_group",
26     "data_privacy_level", "ai_ethics_committee",
27     "survey_source", "data_collection_method"
28 ]
29
30 # D = 1 for partial/full adoption, 0 for none/pilot
31 HIGH_STAGES = ["partial", "full"]
32 df["d_ai_adoption"] = df[D_COL].apply(lambda x: 1 if str(x).lower() in
33     HIGH_STAGES else 0)
34
35 # Y = log(revenue / employees)
36 df["y_log_productivity"] = np.log(df[Y_REVENUE] / df[Y_EMPLOYEES])
37
38 # Identify numeric covariates automatically
39 all_numeric = df.select_dtypes(include=[np.number]).columns.tolist()
40 NUM_COVARIATES = [c for c in all_numeric if c not in set(ID_COLS + [
41     Y_REVENUE, Y_EMPLOYEES])]
42
43 # Save cleaned data
44 save_cols = list(dict.fromkeys(
45     ["d_ai_adoption", "y_log_productivity", D_COL, Y_REVENUE, Y_EMPLOYEES]
46     + [c for c in CAT_COVARIATES if c in df.columns]
47     + NUM_COVARIATES
48 ))
49 df[save_cols].to_csv(os.path.join(OUTPUT_TABLES, "cleaned_data.csv"), index=
50     False)
```

A.2 可视化 (02_visualization.py)


```

1 import pandas as pd, numpy as np, matplotlib.pyplot as plt, seaborn as sns,
  os
2 import warnings; warnings.filterwarnings("ignore")
3
4 PROJECT_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
5 OUTPUT_TABLES = os.path.join(PROJECT_ROOT, "output", "tables")
6 OUTPUT_FIGURES = os.path.join(PROJECT_ROOT, "output", "figures")
7 os.makedirs(OUTPUT_FIGURES, exist_ok=True)
8
9 plt.rcParams.update({"figure.dpi": 150, "font.size": 11,
10     "axes.spines.top": False, "axes.spines.right": False})
11 COLOR = ["#4472C4", "#ED7D31"]
12
13 df = pd.read_csv(os.path.join(OUTPUT_TABLES, "cleaned_data.csv"))
14
15 # Fig 1: AI adoption rate by industry
16 industry_rate = df.groupby("industry")["d_ai_adoption"].mean().sort_values()
17 fig, ax = plt.subplots(figsize=(8, 5))
18 colors = [COLOR[0] if v < 0.5 else COLOR[1] for v in industry_rate.values]
19 ax.barh(industry_rate.index, industry_rate.values, color=colors, edgecolor="
    white")
20 ax.set_xlabel("AI Adoption Rate"); ax.set_title("AI Adoption Rate by
    Industry")
21 for i, (idx, v) in enumerate(industry_rate.items()):
22     ax.text(v + 0.01, i, f"{v:.1%}", va="center", fontsize=8)
23 plt.tight_layout()
24 fig.savefig(os.path.join(OUTPUT_FIGURES, "fig1_ai_adoption_by_industry.pdf")
    , bbox_inches="tight")
25 plt.close()
26
27 # Fig 2: Productivity distribution by adoption status
28 fig, axes = plt.subplots(1, 2, figsize=(10, 4.5))
29 sns.violinplot(data=df, x="d_ai_adoption", y="y_log_productivity",
30     palette=[COLOR[0], COLOR[1]], inner="quartile", ax=axes[0])
31 axes[0].set_xticklabels(["Low Adoption (D=0)", "High Adoption (D=1)"])
32 axes[0].set_ylabel("Log(Revenue / Employees)"); axes[0].set_title("
    Productivity Distribution")
33
34 means = df.groupby("d_ai_adoption")["y_log_productivity"].agg(["mean", "sem"
    ])
35 axes[1].bar([0, 1], means["mean"], yerr=1.96 * means["sem"],
36     color=[COLOR[0], COLOR[1]], capsize=5, edgecolor="white")
37 axes[1].set_xticks([0, 1]); axes[1].set_xticklabels(["Low (D=0)", "High (D
    =1)"])
38 axes[1].set_ylabel("Mean Log Productivity"); axes[1].set_title("Mean $\pm$
    95% CI")
39 diff = means.loc[1, "mean"] - means.loc[0, "mean"]
40 axes[1].annotate(f"Diff = {diff:.3f}", xy=(0.5, 0.95), xycoords="axes
    fraction",
41     ha="center", bbox=dict(boxstyle="round", facecolor="wheat",
    alpha=0.5))
42 plt.tight_layout()
43 fig.savefig(os.path.join(OUTPUT_FIGURES, "fig2_productivity_by_adoption.pdf"
    ), bbox_inches="tight")
44 plt.close()
45
46 # Fig 3: AI adoption by firm size
47 size_order = [s for s in ["Startup", "SME", "Enterprise"] if s in df["
    company_size"].unique()]

```



```

48 size_order += [s for s in df["company_size"].unique() if s not in size_order
    ]
49 ct = pd.crosstab(df["company_size"], df["d_ai_adoption"], normalize="index")
50 ct = ct.reindex([s for s in size_order if s in ct.index])
51 fig, ax = plt.subplots(figsize=(7, 4.5))
52 ct.plot(kind="bar", stacked=True, ax=ax, color=[COLOR[0], COLOR[1]],
    edgecolor="white")
53 ax.set_ylabel("Proportion"); ax.set_title("AI Adoption by Company Size")
54 for i, size in enumerate(ct.index):
55     ax.text(i, ct.loc[size, 1] / 2, f"{ct.loc[size, 1]:.0%}", ha="center",
56         va="center", fontsize=10, color="white", fontweight="bold")
57 plt.tight_layout()
58 fig.savefig(os.path.join(OUTPUT_FIGURES, "fig3_adoption_by_firm_size.pdf"),
    bbox_inches="tight")
59 plt.close()
60
61 # Fig 4: Correlation heatmap
62 key_vars = [c for c in ["y_log_productivity", "d_ai_adoption", "
    ai_adoption_rate",
63     "task_automation_rate", "ai_maturity_score", "innovation_score",
64     "num_ai_tools_used", "employee_satisfaction_score", "
    regulatory_compliance_score"]
65     if c in df.columns]
66 corr = df[key_vars].corr()
67 fig, ax = plt.subplots(figsize=(9, 7))
68 mask = np.triu(np.ones_like(corr, dtype=bool), k=1)
69 sns.heatmap(corr, mask=mask, annot=True, fmt=".2f", cmap="RdBu_r",
70     center=0, square=True, linewidths=0.5, ax=ax)
71 ax.set_title("Correlation Matrix of Key Variables")
72 plt.tight_layout()
73 fig.savefig(os.path.join(OUTPUT_FIGURES, "fig4_correlation_heatmap.pdf"),
    bbox_inches="tight")
74 plt.close()

```

A.3 核心 DML 估计 (03_dml_estimation.py)

```

1 import pandas as pd, numpy as np, os, warnings, time
2 from sklearn.model_selection import KFold
3 from scipy import stats
4 from sklearn.ensemble import RandomForestRegressor
5 warnings.filterwarnings("ignore")
6
7 PROJECT_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
8 OUTPUT_TABLES = os.path.join(PROJECT_ROOT, "output", "tables")
9 os.makedirs(OUTPUT_TABLES, exist_ok=True)
10 np.random.seed(42)
11
12 df = pd.read_csv(os.path.join(OUTPUT_TABLES, "cleaned_data.csv"))
13 Y = df["y_log_productivity"].values
14 D = df["d_ai_adoption"].values
15
16 # Covariates: predetermined firm characteristics
17 COVARIATE_COLS = [
18     "industry", "country", "region", "company_size",
19     "company_age", "company_age_group", "company_founding_year",
20     "survey_year", "quarter",
21     "task_automation_rate", "remote_work_percentage",
22     "regulatory_compliance_score", "data_privacy_level",
23     "ai_ethics_committee", "ai_risk_management_score",

```

```

24     "employee_satisfaction_score", "innovation_score",
25 ]
26 COVARIATE_COLS = [c for c in COVARIATE_COLS if c in df.columns]
27
28 # One-hot encoding; survey_year as categorical fixed effect
29 FORCE_CATEGORICAL = ["survey_year"]
30 numeric_x = [c for c in COVARIATE_COLS if c in df.select_dtypes(include=[np.
    number]).columns]
31 categorical_x = [c for c in COVARIATE_COLS if c in df.select_dtypes(include
    =["object"]).columns]
32 categorical_x = list(set(categorical_x + FORCE_CATEGORICAL))
33 numeric_x = [c for c in numeric_x if c not in categorical_x]
34 df_x = pd.get_dummies(df[COVARIATE_COLS], columns=categorical_x, drop_first=
    True)
35 X = df_x.values.astype(np.float64)
36
37 def dml_cross_fit(Y, D, X, model_y, model_t, n_folds=5, seed=42):
38     """DML with cross-fitting and Neyman orthogonal score"""
39     kf = KFold(n_splits=n_folds, shuffle=True, random_state=seed)
40     n = len(Y)
41     Y_res, D_res = np.zeros(n), np.zeros(n)
42     for train_idx, test_idx in kf.split(X):
43         # Estimate nuisance functions on training fold, compute residuals on
            test fold
44         model_y.fit(X[train_idx], Y[train_idx])
45         model_t.fit(X[train_idx], D[train_idx])
46         Y_res[test_idx] = Y[test_idx] - model_y.predict(X[test_idx])
47         D_res[test_idx] = D[test_idx] - model_t.predict(X[test_idx])
48     # OLS on residuals: tilde_Y = theta * tilde_D
49     theta = np.nansum(D_res * Y_res) / (np.nansum(D_res ** 2) + 1e-10)
50     # Eicker-Huber-White robust standard errors
51     e = Y_res - theta * D_res
52     se = np.sqrt(np.nansum(D_res ** 2 * e ** 2) / ((np.nansum(D_res ** 2) **
        2) + 1e-10))
53     t_val = theta / se
54     p_val = 2 * (1 - stats.norm.cdf(abs(t_val)))
55     return theta, se, theta - 1.96 * se, theta + 1.96 * se, p_val
56
57 # Baseline: Random Forest with K=5
58 rf_y = RandomForestRegressor(n_estimators=100, max_depth=10,
    min_samples_leaf=10,
59                             random_state=42, n_jobs=-1)
60 rf_t = RandomForestRegressor(n_estimators=100, max_depth=10,
    min_samples_leaf=10,
61                             random_state=42, n_jobs=-1)
62 theta, se, ci_l, ci_u, p_val = dml_cross_fit(Y, D, X, rf_y, rf_t, n_folds=5)
63
64 results = pd.DataFrame([{"method": "DML (Random Forest)", "estimate": theta,
65     "std_err": se, "ci_lower": ci_l, "ci_upper": ci_u, "p_value": p_val,
66     "n": len(Y), "n_covariates": X.shape[1], "n_folds": 5}])
67 results.to_csv(os.path.join(OUTPUT_TABLES, "dml_results.csv"), index=False)

```

A.4 稳健性检验 (04_robustness.py)

```

1 import pandas as pd, numpy as np, os, warnings, time
2 from sklearn.model_selection import KFold
3 from scipy import stats
4 from sklearn.ensemble import RandomForestRegressor
5 from sklearn.linear_model import LassoCV

```

```

6 import xgboost as xgb
7 warnings.filterwarnings("ignore")
8
9 PROJECT_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
10 OUTPUT_TABLES = os.path.join(PROJECT_ROOT, "output", "tables")
11 np.random.seed(42)
12
13 df = pd.read_csv(os.path.join(OUTPUT_TABLES, "cleaned_data.csv"))
14
15 def dml_cross_fit(Y, D, X, model_y, model_t, n_folds=5, seed=42):
16     kf = KFold(n_splits=n_folds, shuffle=True, random_state=seed)
17     n = len(Y); Y_res, D_res = np.zeros(n), np.zeros(n)
18     for tr, te in kf.split(X):
19         model_y.fit(X[tr], Y[tr]); Y_res[te] = Y[te] - model_y.predict(X[te])
20         model_t.fit(X[tr], D[tr]); D_res[te] = D[te] - model_t.predict(X[te])
21     theta = np.nansum(D_res * Y_res) / np.nansum(D_res ** 2)
22     e = Y_res - theta * D_res
23     se = np.sqrt(np.nansum(D_res ** 2 * e ** 2) / (np.nansum(D_res ** 2) ** 2))
24     t = theta / se
25     return theta, se, theta - 1.96 * se, theta + 1.96 * se, 2 * (1 - stats.
        norm.cdf(abs(t)))
26
27 COVARIATE_COLS = ["industry", "country", "region", "company_size", "
    company_age",
28     "company_age_group", "company_founding_year", "survey_year", "quarter",
29     "task_automation_rate", "remote_work_percentage", "
    regulatory_compliance_score",
30     "data_privacy_level", "ai_ethics_committee", "ai_risk_management_score",
31     "employee_satisfaction_score", "innovation_score"]
32 COVARIATE_COLS = [c for c in COVARIATE_COLS if c in df.columns]
33
34 def prepare_X(df, extra_exclude=None):
35     cols = [c for c in COVARIATE_COLS if c not in (extra_exclude or [])]
36     cat = [c for c in cols if c in df.select_dtypes(include=["object"]).
        columns]
37     cat = list(set(cat + ["survey_year"]))
38     return pd.get_dummies(df[cols], columns=cat, drop_first=True).values.
        astype(np.float64)
39
40 X_base, Y_base, D_base = prepare_X(df), df["y_log_productivity"].values, df[
    "d_ai_adoption"].values
41
42 results = []
43 # Baseline (loaded from saved results)
44 bl = pd.read_csv(os.path.join(OUTPUT_TABLES, "dml_results.csv"))
45 results.append({"specification": "Baseline (RF, K=5)", "estimate": bl["
    estimate"].values[0],
46     "std_err": bl["std_err"].values[0], "ci_lower": bl["ci_lower"].values
        [0],
47     "ci_upper": bl["ci_upper"].values[0], "p_value": bl["p_value"].values
        [0]})
48
49 # 1a: LASSO with cross-validation
50 ly = LassoCV(cv=5, random_state=42, max_iter=5000)
51 lt = LassoCV(cv=5, random_state=42, max_iter=5000)
52 th, se, cl, cu, pv = dml_cross_fit(Y_base, D_base, X_base, ly, lt)

```

```

53 results.append({"specification": "LASSO (CV)", "estimate": th, "std_err": se
54               ,
55               "ci_lower": cl, "ci_upper": cu, "p_value": pv})
56 # 1b: XGBoost
57 xy = xgb.XGBRegressor(n_estimators=100, max_depth=4, lr=0.1, random_state
58                       =42, n_jobs=-1, verbosity=0)
59 xt = xgb.XGBRegressor(n_estimators=100, max_depth=4, lr=0.1, random_state
60                       =42, n_jobs=-1, verbosity=0)
61 th, se, cl, cu, pv = dml_cross_fit(Y_base, D_base, X_base, xy, xt)
62 results.append({"specification": "XGBoost", "estimate": th, "std_err": se,
63               "ci_lower": cl, "ci_upper": cu, "p_value": pv})
64 # 2: Different numbers of folds K = 2, 10
65 for K in [2, 10]:
66     rf = RandomForestRegressor(n_estimators=100, max_depth=10,
67                               min_samples_leaf=10, random_state=42, n_jobs=-1)
68     th, se, cl, cu, pv = dml_cross_fit(Y_base, D_base, X_base, rf, rf,
69                                       n_folds=K)
70     results.append({"specification": f"K = {K}", "estimate": th, "std_err":
71                   se,
72                   "ci_lower": cl, "ci_upper": cu, "p_value": pv})
73 # 3: Strict treatment definition (full adoption only)
74 D_strict = (df["ai_adoption_stage"].str.lower() == "full").astype(int).
75             values
76 rf = RandomForestRegressor(n_estimators=100, max_depth=10, min_samples_leaf
77                             =10, random_state=42, n_jobs=-1)
78 th, se, cl, cu, pv = dml_cross_fit(Y_base, D_strict, X_base, rf, rf)
79 results.append({"specification": "Strict D (full only)", "estimate": th, "
80               std_err": se,
81               "ci_lower": cl, "ci_upper": cu, "p_value": pv})
82 # 4: Alternative outcome variable
83 if "productivity_change_percent" in df.columns:
84     Y_alt = df["productivity_change_percent"].values
85     X_alt = prepare_X(df, extra_exclude=["productivity_change_percent"])
86     rf = RandomForestRegressor(n_estimators=100, max_depth=10,
87                               min_samples_leaf=10, random_state=42, n_jobs=-1)
88     th, se, cl, cu, pv = dml_cross_fit(Y_alt, D_base, X_alt, rf, rf)
89     results.append({"specification": "Alt Y (productivity_change)", "
90                   estimate": th,
91                   "std_err": se, "ci_lower": cl, "ci_upper": cu, "p_value": pv})
92 pd.DataFrame(results).to_csv(os.path.join(OUTPUT_TABLES, "robustness_results
93                                   .csv"), index=False)

```

A.5 内生性诊断 (05_diagnostics.py)

```

1 import pandas as pd, numpy as np, os, warnings, time
2 from sklearn.model_selection import KFold
3 from scipy import stats
4 from sklearn.ensemble import RandomForestRegressor
5 import matplotlib.pyplot as plt
6 warnings.filterwarnings("ignore")
7
8 PROJECT_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
9 OUTPUT_TABLES = os.path.join(PROJECT_ROOT, "output", "tables")
10 OUTPUT_FIGURES = os.path.join(PROJECT_ROOT, "output", "figures")

```

```

11 os.makedirs(OUTPUT_FIGURES, exist_ok=True); np.random.seed(42)
12 plt.rcParams.update({"figure.dpi": 150, "axes.spines.top": False, "axes.
    spines.right": False})
13 COLOR = ["#4472C4", "#ED7D31"]
14
15 df = pd.read_csv(os.path.join(OUTPUT_TABLES, "cleaned_data.csv"))
16 COVARIATE_COLS = ["industry", "country", "region", "company_size", "
    company_age",
17     "company_age_group", "company_founding_year", "survey_year", "quarter",
18     "task_automation_rate", "remote_work_percentage", "
    regulatory_compliance_score",
19     "data_privacy_level", "ai_ethics_committee", "ai_risk_management_score",
20     "employee_satisfaction_score", "innovation_score"]
21
22 def build_X(df, extra_exclude=None):
23     cols = [c for c in COVARIATE_COLS if c not in (extra_exclude or [])]
24     cat = [c for c in cols if c in df.select_dtypes(include=["object"]).
        columns]
25     cat = list(set(cat + ["survey_year"]))
26     return pd.get_dummies(df[cols], columns=cat, drop_first=True).values.
        astype(np.float64)
27
28 X, Y, D = build_X(df), df["y_log_productivity"].values, df["d_ai_adoption"].
    values
29
30 def dml_cross_fit(Y, D, X, my, mt, n_folds=5, seed=42):
31     kf = KFold(n_splits=n_folds, shuffle=True, random_state=seed)
32     n = len(Y); Yr, Dr = np.zeros(n), np.zeros(n)
33     for tr, te in kf.split(X):
34         my.fit(X[tr], Y[tr]); mt.fit(X[tr], D[tr])
35         Yr[te] = Y[te] - my.predict(X[te]); Dr[te] = D[te] - mt.predict(X[te]
        ])
36     th = np.nansum(Dr * Yr) / (np.nansum(Dr ** 2) + 1e-10)
37     e = Yr - th * Dr
38     se = np.sqrt(np.nansum(Dr ** 2 * e ** 2) / (np.nansum(Dr ** 2) ** 2 + 1e
        -10))
39     pv = 2 * (1 - stats.norm.cdf(abs(th / (se + 1e-10))))
40     return th, se, th - 1.96 * se, th + 1.96 * se, pv, Dr, Yr
41
42 # --- 1. Common support (overlap) ---
43 rf_prop = RandomForestRegressor(n_estimators=100, max_depth=8,
    min_samples_leaf=10, random_state=42, n_jobs=-1)
44 rf_prop.fit(X, D); propensity = rf_prop.predict(X)
45
46 fig, axes = plt.subplots(1, 2, figsize=(9, 3.5))
47 for dv, c, lb in [(0, COLOR[0], "Low (D=0)"), (1, COLOR[1], "High (D=1)"]]:
48     axes[0].hist(propensity[D == dv], bins=50, alpha=0.6, color=c, label=lb,
        density=True)
49 axes[0].axvline(0.1, ls="--", color="gray", alpha=0.5)
50 axes[0].axvline(0.9, ls="--", color="gray", alpha=0.5)
51 axes[0].set_xlabel("P(D=1|X)"); axes[0].set_ylabel("Density"); axes[0].
    legend()
52 extreme_0 = np.mean((D == 0) & (propensity > 0.9))
53 extreme_1 = np.mean((D == 1) & (propensity < 0.1))
54 axes[1].bar(["D=0, prop>0.9", "D=1, prop<0.1"], [extreme_0, extreme_1],
    color=COLOR)
55 axes[1].set_ylabel("Proportion")
56 plt.tight_layout()
57 fig.savefig(os.path.join(OUTPUT_FIGURES, "fig5_overlap.pdf"), bbox_inches="
    tight")

```

```

58 plt.close()
59
60 # --- 2. Placebo test (shuffle Y) ---
61 placebo_pvals = []
62 for i in range(5):
63     Ys = np.random.permutation(Y)
64     th, se, _, _, pv, _, _ = dml_cross_fit(Ys, D, X,
65         RandomForestRegressor(50, 8, 15, random_state=42+i, n_jobs=-1),
66         RandomForestRegressor(50, 8, 15, random_state=42+i, n_jobs=-1))
67     placebo_pvals.append(pv)
68 fpr = np.mean(np.array(placebo_pvals) < 0.05)
69
70 # --- 3. Covariate balance ---
71 _, _, _, _, _, Dr, _ = dml_cross_fit(Y, D, X,
72     RandomForestRegressor(100, 10, 10, random_state=42, n_jobs=-1),
73     RandomForestRegressor(100, 10, 10, random_state=42, n_jobs=-1))
74 num_vars = df[COVARIATE_COLS].select_dtypes(include=[np.number]).columns.
75     tolist()
76 bal = pd.DataFrame([(v, abs(np.corrcoef(D, df[v].values)[0,1]),
77     abs(np.corrcoef(Dr, df[v].values)[0,1])) for v in
78     num_vars],
79     columns=["var", "raw", "resid"]).sort_values("raw",
80     ascending=False)
81
82 fig, ax = plt.subplots(figsize=(7, 4))
83 ax.scatter(range(len(bal)), bal["raw"], color=COLOR[0], label="|corr(D,X)|",
84     alpha=0.7)
85 ax.scatter(range(len(bal)), bal["resid"], color=COLOR[1], label="|corr(D_res
86     ,X)|", alpha=0.7, marker="D")
87 ax.axhline(0.1, color="gray", ls="--", alpha=0.5)
88 ax.set_xticks(range(len(bal))); ax.set_xticklabels(bal["var"], rotation=45,
89     ha="right", fontsize=8)
90 ax.set_ylabel("|Correlation|"); ax.legend()
91 plt.tight_layout()
92 fig.savefig(os.path.join(OUTPUT_FIGURES, "fig6_covariate_balance.pdf"),
93     bbox_inches="tight")
94 plt.close()
95
96 # --- 4. Sensitivity to unobserved confounding ---
97 sens = []
98 for rho in np.arange(0, 0.61, 0.1):
99     U = rho * Dr + np.sqrt(1 - rho**2) * np.random.randn(len(Y))
100     U = (U - U.mean()) / U.std()
101     th, se, _, _, pv, _, _ = dml_cross_fit(Y, D, np.column_stack([X, U]),
102         RandomForestRegressor(100, 10, 10, random_state=42, n_jobs=-1),
103         RandomForestRegressor(100, 10, 10, random_state=42, n_jobs=-1))
104     sens.append({"rho": rho, "est": th, "se": se, "p": pv})
105 sens_df = pd.DataFrame(sens)
106
107 fig, ax1 = plt.subplots(figsize=(6.5, 4))
108 ax1.errorbar(sens_df["rho"], sens_df["est"], yerr=1.96*sens_df["se"],
109     fmt="-o", color=COLOR[0], capsize=4, label="ATE +/- 95% CI")
110 ax1.axhline(y=0, color="gray", ls="--", alpha=0.7)
111 ax1.set_xlabel("Confounder Strength  $\rho$ "); ax1.set_ylabel("ATE"); ax1.
112     legend(loc="upper left")
113 ax2 = ax1.twinx()
114 ax2.plot(sens_df["rho"], sens_df["p"], "s--", color=COLOR[1], markersize=5,
115     alpha=0.7, label="p-value")
116 ax2.axhline(y=0.05, color=COLOR[1], ls=":", alpha=0.5)

```

```

108 ax2.set_ylabel("p-value", color=COLOR[1]); ax2.tick_params(axis="y",
    labelcolor=COLOR[1])
109 ax2.legend(loc="upper right")
110 plt.tight_layout()
111 fig.savefig(os.path.join(OUTPUT_FIGURES, "fig7_sensitivity.pdf"),
    bbox_inches="tight")
112 plt.close()
113
114 # Save results
115 pd.DataFrame([("Overlap", f"Extreme {extreme_0:.3f}/{extreme_1:.3f}"),
116     ("Placebo", f"FPR {fpr:.1%}"), ("Balance", f"Mean |corr|={bal['resid'].
    mean():.3f}"),
117     ("Sensitivity", "Critical rho>0.6")], columns=["test", "detail"]
118 ).to_csv(os.path.join(OUTPUT_TABLES, "diagnostics_results.csv"), index=False)
119 sens_df.to_csv(os.path.join(OUTPUT_TABLES, "sensitivity_analysis.csv"),
    index=False)

```

A.6 异质性分析 (06_heterogeneity_analysis.py)

```

1 import pandas as pd, numpy as np, os, warnings
2 from sklearn.model_selection import KFold
3 from scipy import stats
4 from sklearn.ensemble import RandomForestRegressor
5 import matplotlib.pyplot as plt
6 from concurrent.futures import ThreadPoolExecutor, as_completed
7 warnings.filterwarnings("ignore")
8
9 PROJECT_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
10 OUTPUT_TABLES = os.path.join(PROJECT_ROOT, "output", "tables")
11 OUTPUT_FIGURES = os.path.join(PROJECT_ROOT, "output", "figures")
12 os.makedirs(OUTPUT_FIGURES, exist_ok=True); np.random.seed(42)
13 plt.rcParams.update({"figure.dpi": 150, "axes.spines.top": False, "axes.
    spines.right": False})
14
15 df = pd.read_csv(os.path.join(OUTPUT_TABLES, "cleaned_data.csv"))
16 COVARIATE_COLS = ["industry", "country", "region", "company_size", "
    company_age",
17     "company_age_group", "company_founding_year", "survey_year", "quarter",
18     "task_automation_rate", "remote_work_percentage", "
    regulatory_compliance_score",
19     "data_privacy_level", "ai_ethics_committee", "ai_risk_management_score",
20     "employee_satisfaction_score", "innovation_score"]
21
22 cat_cols = [c for c in COVARIATE_COLS if c in df.select_dtypes(include=["
    object"])]
23 if "survey_year" in COVARIATE_COLS and "survey_year" not in cat_cols:
24     cat_cols.append("survey_year")
25 X_full = pd.get_dummies(df[COVARIATE_COLS], columns=cat_cols, drop_first=
    True).values.astype(np.float64)
26 X_COLS = pd.get_dummies(df[COVARIATE_COLS], columns=cat_cols, drop_first=
    True).columns.tolist()
27 Y, D = df["y_log_productivity"].values, df["d_ai_adoption"].values
28 stages = df["ai_adoption_stage"].values
29
30 def dml_cross_fit(Y, D, X, my, mt, n_folds=5, seed=42):
31     if len(Y) < 200 or np.sum(D > 0.5) < 30 or np.sum(D < 0.5) < 30:
32         return np.nan, np.nan, np.nan, np.nan, np.nan, None, None
33     kf = KFold(n_splits=n_folds, shuffle=True, random_state=seed)

```



```

34     n = len(Y); Yr, Dr = np.zeros(n), np.zeros(n)
35     for tr, te in kf.split(X):
36         try:
37             my.fit(X[tr], Y[tr]); mt.fit(X[tr], D[tr])
38             Yr[te] = Y[te] - my.predict(X[te]); Dr[te] = D[te] - mt.predict(
39                 X[te])
40         except: return np.nan, np.nan, np.nan, np.nan, np.nan, None, None
41     th = np.nansum(Dr * Yr) / (np.nansum(Dr ** 2) + 1e-10)
42     se = np.sqrt(np.nansum(Dr ** 2 * (Yr - th * Dr) ** 2) / (np.nansum(Dr **
43         2) ** 2 + 1e-10))
44     pv = 2 * (1 - stats.norm.cdf(abs(th / (se + 1e-10))))
45     return th, se, th - 1.96 * se, th + 1.96 * se, pv, Dr, Yr
46
47 def rf(seed): return RandomForestRegressor(80, 8, 15, random_state=seed,
48     n_jobs=-1)
49
50 # --- 1. Multi-value treatment (vs. "none" as reference) ---
51 mv_results = []
52 for label, treat, ctrl in [("Pilot vs None", "pilot", "none"),
53     ("Partial vs None", "partial", "none"), ("Full vs None", "full", "none")
54     ]:
55     mask = (stages == treat) | (stages == ctrl)
56     idx = np.where(mask)[0]; D_sub = (stages[idx] == treat).astype(float)
57     nf = min(5, max(2, min(D_sub.sum(), (1-D_sub).sum()) // 100))
58     th, se, _, _, pv, _, _ = dml_cross_fit(Y[idx], D_sub, X_full[idx], rf
59         (42), rf(43), n_folds=nf)
60     mv_results.append({"comparison": label, "ATE": th, "SE": se, "p_value":
61         pv, "n": len(idx)})
62 pd.DataFrame(mv_results).to_csv(os.path.join(OUTPUT_TABLES, "
63     multivalue_results.csv"), index=False)
64
65 # --- 2. Heterogeneous treatment effects by subgroup ---
66 def make_configs():
67     cfg = []
68     for ind in df["industry"].unique():
69         cfg.append((X_COLS, "industry", ind, df["industry"].values == ind,
70             True))
71     for sz in df["company_size"].unique():
72         cfg.append((X_COLS, "company_size", sz, df["company_size"].values ==
73             sz, True))
74     for yr in sorted(df["survey_year"].unique()):
75         cfg.append((X_COLS, "survey_year", str(yr), df["survey_year"].values
76             == yr, True))
77     med = df["task_automation_rate"].median()
78     cfg.append((X_COLS, "automation", "high", df["task_automation_rate"].
79         values >= med, False))
80     cfg.append((X_COLS, "automation", "low", df["task_automation_rate"].
81         values < med, False))
82     return cfg
83
84 def run_hte(args):
85     xcols, dim, lev, mask, do_exclude = args
86     idx = np.where(mask)[0]
87     exc = [xcols.index(c) for c in xcols if c.startswith(dim + "_")] if
88         do_exclude else []
89     Xs = np.delete(X_full[idx], exc, axis=1) if exc else X_full[idx]
90     nt, nc = int(D[idx].sum()), int((1-D[idx]).sum())
91     if nt < 50 or nc < 50:
92         return {"dim": dim, "level": lev, "ATE": np.nan, "SE": np.nan, "p":
93             np.nan, "n": len(idx)}

```

```

80     nf = min(3, max(2, min(nt, nc) // 50))
81     th, se, _, _, pv, _, _ = dml_cross_fit(Y[idx], D[idx], Xs, rf(42), rf
      (43), n_folds=nf)
82     return {"dim": dim, "level": lev, "ATE": th, "SE": se, "p": pv, "n": len
      (idx)}
83
84 hte_res = []
85 with ThreadPoolExecutor(max_workers=3) as ex:
86     for f in as_completed([ex.submit(run_hte, c) for c in make_configs()]):
87         hte_res.append(f.result())
88 hte_df = pd.DataFrame(hte_res)
89 hte_df.to_csv(os.path.join(OUTPUT_TABLES, "hte_results.csv"), index=False)
90
91 # --- 3. Forest plot ---
92 plot_df = hte_df.dropna(subset=["ATE"]).copy()
93 if len(plot_df) > 0:
94     plot_df["label"] = plot_df["dim"] + ": " + plot_df["level"]
95     plot_df = plot_df.sort_values("ATE")
96     fig, ax = plt.subplots(figsize=(8, max(4, len(plot_df) * 0.35)))
97     for i, (_, r) in enumerate(plot_df.iterrows()):
98         c = "#ED7D31" if r["p"] < 0.05 else "#4472C4"
99         ax.errorbar(r["ATE"], i, xerr=1.96*r["SE"], fmt="o", color=c,
      capsize=4)
100     ax.axvline(x=0, color="gray", ls="--", alpha=0.7)
101     ax.set_yticks(range(len(plot_df)))
102     ax.set_yticklabels(plot_df["label"], fontsize=9)
103     ax.set_xlabel("ATE (95% CI)"); ax.set_title("Heterogeneous Treatment
      Effects")
104     plt.tight_layout()
105     fig.savefig(os.path.join(OUTPUT_FIGURES, "fig8_hte_forest.pdf"),
      bbox_inches="tight")
106     plt.close()

```